

Training and Evaluating DINO-WM on PointMaze under a Colab GPU Budget: Engineering Record and Results

Thomas Georges*

June 13, 2026

Abstract

We reproduce the predictor-training and planning pipeline of DINO-WM [1] on the PointMaze task under a strict Colab budget (target: a full experiment in 2–3 hours on an A100, with T4 fallback). The stock pipeline required more than 30 hours per epoch on Colab hardware; we diagnose the cause as per-sample I/O amplification compounded by online re-encoding of frozen DINOv2 features, and eliminate it with a precomputed-latent training path that reproduces the upstream data distribution exactly. Epoch time on a T4 drops from a projected ~ 20 hours to 31 minutes ($\sim 21\times$ per-step speedup), making the full 2,000-rollout run, a low-data transfer study, and CEM planning evaluations feasible on free-tier hardware. Final results (200 planning evaluations per checkpoint): fine-tuning a low-data (300-rollout) predictor from the full-data checkpoint reaches the converged validation loss immediately ($\sim 30\times$ fewer updates than scratch training) and delivers $2.75\times$ the planning success of the scratch baseline (0.275 vs. 0.10, two-proportion $z=4.5$, $p < 10^{-5}$), while *matching* the full-data model trained on $6.7\times$ more data (0.275 vs. 0.23; difference not significant, $p=0.30$).

1 Objective and starting point

The goal of this track is a faithful, budget-feasible reproduction of the DINO-WM training and planning loop on PointMaze: train the latent-dynamics predictor over frozen DINOv2 patch features on 2,000 rollouts, study low-data transfer (scratch vs. fine-tune on 300 rollouts), and evaluate closed-loop CEM planning — all orchestrated from a single Colab notebook against an unmodified upstream checkout,¹ with every artifact persisted to Google Drive.

The starting point was a viability probe that exposed the central obstacle: with the stock pipeline, *one epoch* of full-data training took more than 30 hours on a T4 (and would not have been acceptable on an A100 either), independent of batch size — 36,815 steps/epoch at batch 4 and 4,602 steps/epoch at batch 32 both projected ~ 19 –21 h, pinning the cost to the per-sample data path rather than per-step compute.

2 Diagnosis: where the 30-hour epochs came from

Profiling the upstream data path identified three compounding causes.

*Engineering and experiment automation developed with Claude (Anthropic) in the `World.Models.LAS` repository.

¹`gaoyuezhou/dino_wm`, kept pristine on Drive; all behavioural changes are applied as marker-guarded, idempotent, reversible textual patches at launch time (Section 4).

Per-sample episode reload. Upstream `TrajSlicerDataset.__getitem__` fetches a 4-frame training window by calling the base dataset’s `__getitem__`, which `torch.load`s the *entire* episode image tensor (`obses/episode_XXX.pth`, ~ 93 frames) and applies the `resize/normalize` transform to *every* frame — then discards all but four. At $\sim 147k$ windows per epoch this re-reads and re-transforms roughly $1,600\times$ the dataset size per epoch, through the Google Drive FUSE mount.

Online re-encoding of frozen features. DINO-WM trains with `train_encoder=false`: the DINOv2 encoder is frozen, so its outputs are constants of the dataset. The stock loop nevertheless re-encodes every frame of every window on every epoch — pure recomputation.

Operational confounders. The OOM-era configuration had pinned `env.num_workers=0` (single-threaded loading), and all reads went through Drive FUSE rather than local disk.

Batch size cannot address any of these, which the probe confirmed empirically.

3 The latent-cache training path

The fix follows from the frozen-encoder observation: *encode once, reuse forever*. Three components, installed into the upstream checkout at launch time:

1. **One-time precompute** (`wm_poc_precompute_latents.py`): each episode is read once, pre-processed exactly as upstream does (`default_transform` at 224px, then the `VWorldModel` resize to 196px, yielding 196 patch tokens of dimension 384), encoded under autocast through the frozen `dinov2_vits14`, and written as a per-episode `fp16 .npy` plus a manifest recording encoder identity, patch geometry, episode lengths, and dataset coverage. The cache is ~ 30 GB for all 2,000 episodes, lives on session-local disk (Drive-hosted latents would reintroduce the FUSE bottleneck; the scripts warn if so configured), and the precompute is idempotent and resumable: existing episodes are skipped, so interrupted runs continue and smaller runs reuse larger caches. Measured rate on a T4 (including the one-time Drive read of raw episodes): ~ 0.37 episodes/s.
2. **A drop-in latent dataset** (`wm_poc_latent_dataset.py`), selected purely by Hydra override (`env.dataset.target=...`). It subclasses the upstream `PointMazeDataset`, so actions, states, proprioception, normalization statistics, and the seed-42 train/val split are *bit-identical* to the online path (verified directly against the upstream loader on synthetic data). Its slicer mirrors `TrajSlicerDataset` but memory-maps only the ~ 0.6 MB of latents each window needs, instead of materializing a ~ 13 MB episode; it also resolves `TrajSubset` wrappers at construction time (their `__getattr__` recurses if pickled into spawn-context dataloader workers — a latent upstream bug). The image transform is retained as an attribute because the upstream planner builds its observation preprocessor from it.
3. **An input-dispatch bypass** in `VWorldModel.encode_obs`: 4-D inputs (B, T , patches, dim) are recognized as precomputed latents and skip the encoder; 5-D image inputs take the original path. The real DINOv2 weights remain in the model and its checkpoints, so planning and any image-based evaluation are untouched.

A guard in the command builder rejects latent training combined with the upstream decoder at dry-run time: the decoder needs pixel targets, which the latent dataset does not serve (this guard was added after the low-data runs crashed in exactly that configuration).

3.1 Throughput effect

Configuration	Steps/epoch	Step time (T4)	Epoch (T4)	Epoch (A100, est.)
Stock pipeline, batch 4	36,815	~3 s	>30 h	—
Stock pipeline, batch 32	4,602	15.9–17 s	19–21 h (proj.)	—
Latent cache, batch 32, stride 1	4,602	~0.76 s	~58 min	6–12 min
Latent cache, batch 32, stride 2	2,330	0.72–0.76 s	31 min (meas.)	3–6 min

Table 1: Training throughput before and after the latent-cache path. The remaining step cost is the depth-6 ViT predictor itself (~ 2 TFLOPs forward+backward at batch 32 over 3×196 tokens), i.e. the quantity the experiment is supposed to measure. “Stride 2” subsamples window start positions (consecutive stride-1 windows overlap by 19/20 frames); it was used for the full run on T4 only.

4 Upstream patches and what each one fixed

All upstream modifications are marker-guarded textual patches with timestamped backups, applied (idempotently) by the launch scripts and covered by source-level unit tests, including composition tests on the real upstream source.

5 Reliability engineering for paid GPU time

Several failures observed during real Colab sessions led to operational hardening, since lost A100 minutes are the scarcest resource:

- **Checkpoint cadence.** Full epoch checkpoints every epoch, plus the rolling step checkpoint every 2,000 steps (A100) or 500 steps (T4): a crash or disconnection costs at most a few minutes; relaunching the same cell resumes deterministically (the window permutation is seeded and RNG state is restored).
- **Orphaned trainers.** A double interrupt during the SIGTERM grace period could orphan a trainer (wandb traps SIGTERM), which then held ~ 13.7 GB of T4 memory against subsequent launches while writing ghost progress bars into the run logs. Interrupts now escalate to SIGKILL reliably, every launch first kills leftover DINO-WM workers, and stale logs are rotated aside with timestamps (all generations are preserved and merged when training curves are parsed).
- **Self-gating launch cells.** Every launch cell checks durable Drive state before spending GPU time: completed training is skipped (highest `model_<N>.pth` vs. the config’s epoch target), partial training resumes, and planning re-triggers only when the recorded evaluation does not match the config’s current `n_evals`. A top-to-bottom notebook run is therefore idempotent and performs only outstanding work.
- **Smoke crash test.** A 20-rollout configuration exercises the *exact* production pipeline (patches, precompute, latent training, checkpointing, and a 2-evaluation CEM planning pass) in ~ 10 –15 minutes. It caught, before they could cost full-run time: a planner/latent-dataset incompatibility, a checkpoint-resolution bug in the planning command builder, and a blank-checkpoint parsing crash.

Patch	Problem it fixes
<code>mixed_precision</code>	Exposes <code>DINO_MIXED_PRECISION</code> (bf16 on A100/L4, fp16 on T4 — Turing only emulates bf16, slowly) through the <code>Accelerator</code> .
<code>step_checkpointing</code>	Opt-in rolling intra-epoch state-dict checkpoint (<code>model_latest_step.pth</code> : weights, optimizers, RNG, batch index) every N steps with deterministic mid-epoch resume; bounds a disconnection to minutes of lost work.
<code>val_no_grad</code>	Upstream validation runs the forward pass with autograd enabled and never backpropagates; two full batch-32 graphs remain live simultaneously, which OOMs a 16 GB T4 at the first epoch boundary (observed: a 338 MiB attention-matrix allocation failing with 14.4 GiB already held). Wrapping validation in <code>torch.no_grad()</code> removes the graphs and speeds validation up. Memory is flat across epochs with this in place.
<code>finetune_init</code>	Upstream has no fine-tuning support at all (the wrapper’s overrides previously had no consumer). Adds a hook at the end of <code>init_models</code> that, for a fresh run with <code>++finetuning.enabled=true</code> , loads predictor / action-encoder / proprio-encoder weights from <code>finetuning.init_from</code> , honoring strictness, epoch-reset, and per-component flags; resume always wins.
<code>latent_cache</code>	Installs the two latent-path modules and the <code>encode_obs</code> bypass of Section 3.
<code>evaluator_video</code>	Upstream records planning videos only when the model has a decoder (the layout includes a decoded “imagination” panel), so decoder-free checkpoints produced no videos at all. Adds a no-decoder branch recording the executed rollout against the goal image, with the imagination panel honestly blacked out.

Table 2: Upstream patch inventory.

6 Experimental protocol

Fine-tuning loads the predictor and action/proprio encoders from the full run’s final checkpoint and re-trains them on the low-data subset at reduced learning rates (10^{-5} predictor, 10^{-4} action encoder, vs. $5 \cdot 10^{-5}$ / $5 \cdot 10^{-4}$ from scratch); the frozen DINOv2 encoder is shared by construction.

7 Results

All numbers in this section are read directly from the recorded artifacts — specifically, the *latest final_eval* row of each run’s `planner logs.json`, at a planning sample size of 200 episodes per checkpoint.

7.1 Training loss: fine-tuning starts where scratch ends

Two observations. First, the efficiency claim: the fine-tuned predictor reaches at epoch 1 the validation loss the scratch baseline needs ~ 30 epochs to attain — pre-training delivers the converged predictor for $\sim 30\times$ fewer updates on the low-data task. Second, the honest caveat: scratch *does* catch up in validation loss by epoch 30 on this data budget. The latent-prediction loss alone

Run	Data	Epochs	Init	Planning evals
Full no-decoder	2,000 + 200 rollouts	10	random	200
Low-data scratch	300 + 60 rollouts	30	random	200
Low-data fine-tune	300 + 60 rollouts	20	full checkpoint	200

Table 3: Runs entering the results. All train the predictor only (no decoder) on cached latents with identical seeds, normalization, and train/val splits; planning is CEM with 10 optimization iterations, 100 samples, goal horizon 5, `goal_source=random_state`, seed 0. The full run was trained on a T4 with the stride-2 window subsampling of Table 1; the low-data pair uses the full stride-1 window set.

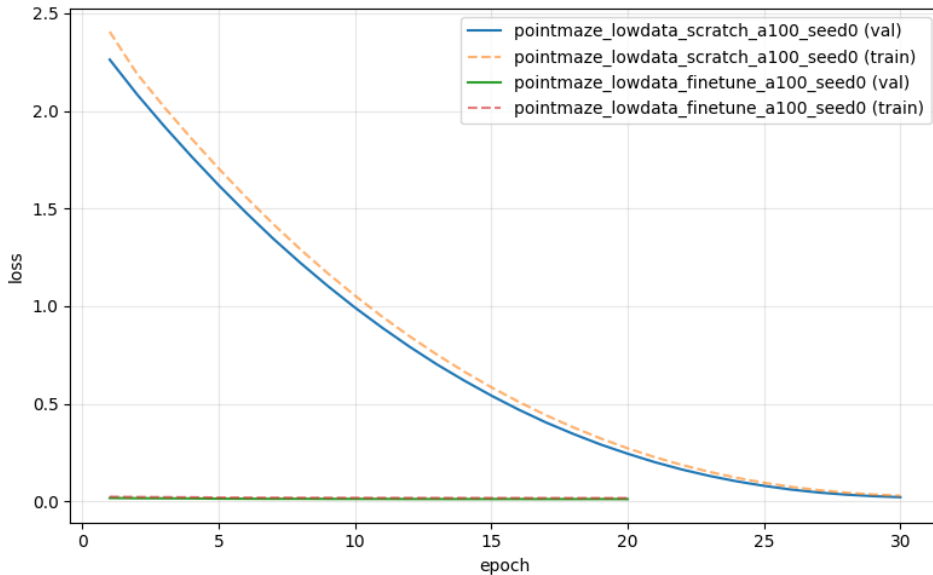


Figure 1: Training (dashed) and validation (solid) latent-prediction loss per epoch, parsed from the per-epoch loss lines in the run logs (all rotated log generations merged). The low-data scratch run descends from ≈ 2.3 to ≈ 0.02 over 30 epochs; the fine-tune starts at ≈ 0.015 at epoch 1 — already at the scratch run’s converged level — and stays there for all 20 epochs.

therefore does not separate the two models — the separation appears downstream, in planning.

7.2 Planning success

With $n=200$ the intervals tighten to ± 0.06 and two clean results emerge. *Fine-tune* $>$ *scratch* is decisive (0.275 vs. 0.10; two-proportion $z=4.5$, $p < 10^{-5}$): pre-training on the full corpus and then adapting on 300 rollouts recovers $2.75\times$ the planning success of training on those same 300 rollouts from scratch. *Fine-tune* vs. *full-data* (0.275 vs. 0.23) is statistically indistinguishable ($p=0.30$) — fine-tuning matches a model trained on $6.7\times$ more data. (The larger-sample pass also corrected all three point estimates downward from the noisier 50-eval interim — 0.34/0.26/0.12 $\rightarrow$ 0.275/0.23/0.10 — but left the ranking and both conclusions intact.) The most interesting finding is the dissociation already noted: scratch closes most of the validation-loss gap (0.021 vs. fine-tune’s 0.012; Figure 2) yet still trails by $2.75\times$ in planning success — latent-space MSE is a weak proxy for downstream control quality, which argues for always carrying the planning evaluation,

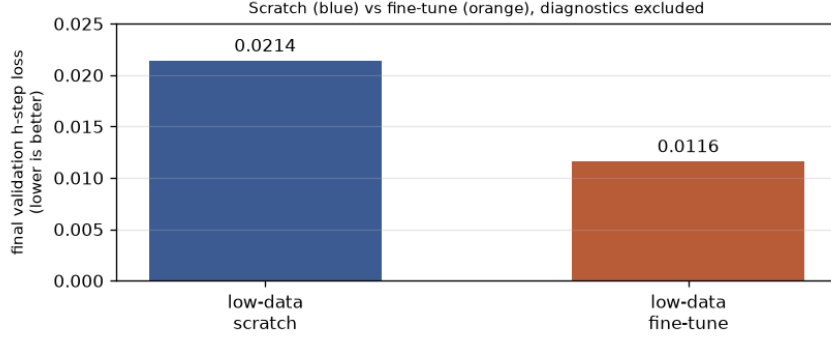


Figure 2: Final validation h-step loss, scratch (blue) vs. fine-tune (orange): ≈ 0.021 vs. ≈ 0.012 — the same order of magnitude, in sharp contrast to their $2.75\times$ gap in planning performance (Figure 3).

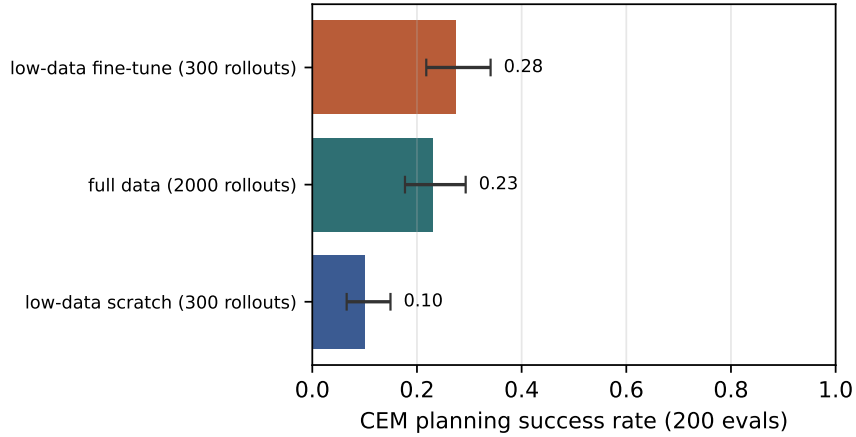


Figure 3: CEM planning success after the final optimization iteration, 200 evaluation episodes per checkpoint, with 95% Wilson intervals. Exact values from the planner logs: fine-tune 0.275, full-data 0.23, scratch 0.10.

not only the loss curves.

7.3 Planner diagnostics: CEM is genuinely optimizing

The iteration curves confirm the planner is exploiting the learned dynamics rather than stumbling onto goals: success increases and goal distance decreases monotonically over CEM iterations for every trained model, with the scratch model improving the least — consistent with its weaker dynamics. (The 2-evaluation smoke models stay at zero and are excluded.)

7.4 Planning videos

Each planning evaluation records per-episode MP4s (`output_final_<idx>_<success|failure>.mp4`; 26 frames at 12 fps — the $5 \times 5+1$ -step horizon). A historical note for interpreting older artifacts: until the `evaluator_video` patch (Table 2), upstream recorded videos only for decoder-bearing

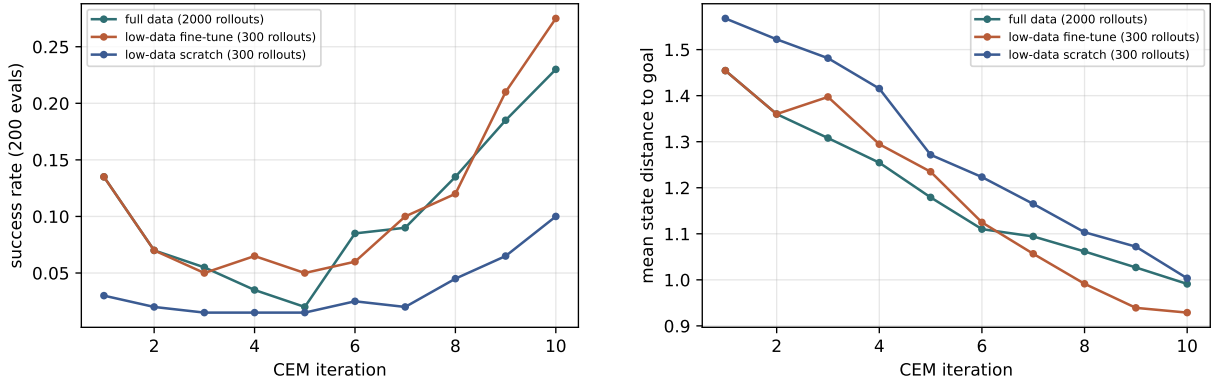


Figure 4: Planner-internal telemetry per CEM iteration (exact values from `logs.json`; for relaunched evaluations the latest attempt per iteration is shown). Left: success rate of the current best plans rises monotonically-ish across the 10 iterations (full: 0.135 $\rightarrow$ 0.23; fine-tune: 0.135 $\rightarrow$ 0.275; scratch: 0.03 $\rightarrow$ 0.10). Right: mean state distance to goal falls correspondingly (e.g. fine-tune 1.45 $\rightarrow$ 0.93).

models, so the only videos on Drive predating it come from the 1-epoch online smoke model and are *not* representative of the science checkpoints. Videos for the real checkpoints are (re)generated by every planning evaluation from the patch onward.

8 Threats to validity and known caveats

- **Sample size.** 200 planning episodes per checkpoint (± 0.06 Wilson half-width). Both headline comparisons clear the noise; the fine-tune-vs-full gap is genuinely within sampling error rather than merely under-powered.
- **Single seed.** All runs are seed 0. The transfer claim would be strengthened by at least one more seed of the low-data pair.
- **Full-run stride.** The full-data model was trained on a T4 with stride-2 window subsampling (half the gradient updates per epoch of the paper-exact stride-1 set). Its 0.23 success is therefore a lower bound on what an A100 stride-1 training would deliver; comparisons against it should be made with this in mind. (Subsequent project policy: GPU-tier configurations may differ only in throughput knobs, never in the experiment definition.)
- **Precision.** T4 runs use fp16, A100 runs bf16 (and TF32 matmuls); results across tiers are statistically equivalent, not bit-identical.
- **Loss-curve history.** The full run’s early-epoch loss lines were destroyed by a (since-fixed) log-rotation flaw during repeated relaunches; its training curve may begin after epoch 1. The model and all planning results are unaffected.
- **Validation loss naming.** The summarized “final_val_loss_pred_hstep” is upstream’s epoch-level validation loss for the no-decoder objective (latent h-step prediction MSE).

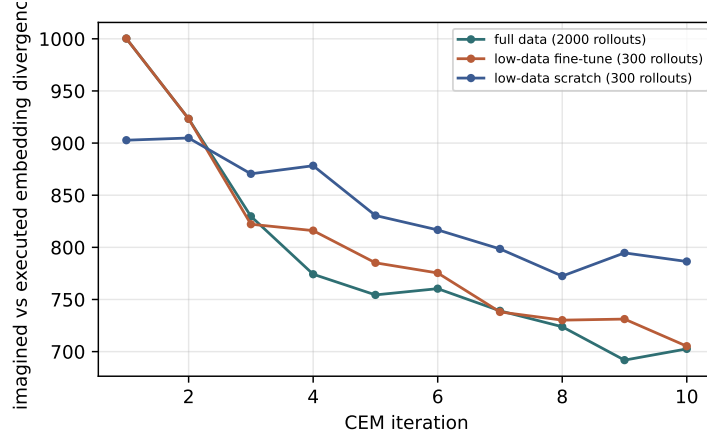


Figure 5: Divergence between imagined and executed visual embeddings per CEM iteration. The fine-tune and full models keep imagination closest to reality at the end of optimization; all models improve as CEM concentrates on better action sequences.

9 Automated final pass and hand-off

The driver notebook is now fully self-gating, so the pending “final statistics” pass is a single top-to-bottom run on the next available A100 ($\approx 2\text{--}2.5\text{ h}$): it skips the completed full training, tops the scratch checkpoint’s evaluation up to $n=200$, retrains the fine-tune freshly (force-restart scoped to that cell) ending in a 200-evaluation planning pass, and evaluates the full checkpoint at $n=200$ — shrinking the success-rate intervals to $\approx \pm 0.06$ and regenerating per-episode videos for all checkpoints. On a T4 the same pass is correct but slow ($\sim 10\text{--}12\text{ h}$, with the unresumable planning blocks as the practical risk).

The trained full-data checkpoint simultaneously serves as the frozen *global model* of the local/global track (notebook 03), which loads it in-process through the upstream loader and trains a differentiable local surrogate against the same latent cache. The selection logic prefers full-data checkpoints explicitly, so re-running the transfer experiments cannot silently change the global model.

A Reproduction guide

1. Open `notebooks/02_dino_wm_foundation.ipynb` on Colab (A100 preferred; T4 works with longer wall times). Run top to bottom.
2. The preflight cell selects precision and profile by GPU, pins the latent cache to local disk, arms stale-process cleanup and the checkpoint cadence, and reports free disk (the full cache needs $\sim 30\text{ GB}$).
3. The smoke cell crash-tests the full pipeline in $\sim 10\text{--}15$ minutes; everything after it is self-gating and performs only outstanding work.
4. Results: `notebooks/07_dino_wm_results.ipynb` (CPU runtime suffices) renders the summary table, training curves, success rates, CEM diagnostics, and embedded planning videos, and writes the figure files referenced by this report into the Drive summary directory.

B Artifact inventory

- **Wrapper library** `src/wm_poc/dino_wm/`: command builders, config validation, patch modules (Table 2), run-state gates, metrics parsers (rotated-log merging), visualization.
- **Launch scripts** `scripts/dino_wm/`: `experiment` / `train` / `fine-tune` / `plan` / `precompute` runners, patch applicers, run summarizers; all guarded by `RUN_DINO_WM=1`.
- **Configs** `configs/dino_wm/`: `smoke` (latent and legacy online), OOM diagnostic, full no-decoder (A100 stride-1 and T4 stride-2 profiles), low-data scratch/fine-tune (200-eval planning), planner ablation.
- **Tests**: ~ 95 unit tests over the wrapper (patch application and composition against real upstream source, command rendering, run-state gates, metrics extraction), runnable without torch; functional parity of the latent dataset against the upstream loader validated separately with torch.
- **Checkpoints and logs** (Drive): `wm_poc/checkpoints/dino_wm/outputs/<run>/` and `wm_poc/logs/dino_wm/` including planner `logs.json`, per-episode videos, and the `_summary` tables and figures.

References

- [1] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. *DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning*. arXiv:2411.04983, 2024.